

■ RAG Implementation

Complete Change Log & Frontend Integration Guide

Backend Changes Summary — 4 Files Modified

File	Kya Badla
services/openai_service.py	generate_embedding() method add kiya
services/ai_automation.py	search_similar_chunks() + get_chunks_by_document() add kiye
schemas/ai_automation.py	3 nayi search schemas add kiye
api/ai.py	Auto-embed on chunk create + 2 nayi endpoints
requirements.txt	numpy + pgvector add kiye

■ RAG Pipeline — Ab Kaisa Kaam Karta Hai

PEHLE (Incomplete)

Frontend -> POST /ai/knowledge/chunks (embedding manually bhejna padta tha) -> DB -> [X] Search nahi t ha

AB (Complete)

```
+-----+
|          INDEXING PHASE (Admin karta hai)          |
|  |                                                  |
| Admin -> POST /ai/knowledge/documents -> Document bana |
| Admin -> POST /ai/knowledge/chunks    -> Server auto- |
|      (sirf content bhejo)              embed karta hai |
|                                          -> pgvector DB mein |
|                                          store                |
+-----+

+-----+
|          SEARCH PHASE (Koi bhi user)              |
|  |                                                  |
| User query -> POST /ai/knowledge/search             |
|  |                                                  |
| Server: query ko embed karta hai (Gemini)           |
|  |                                                  |
| pgvector: cosine distance se top-5 chunks dhundta hai |
|  |                                                  |
| Response: ranked chunks + similarity scores         |
+-----+
```

■ Change 1 — services/openai_service.py

Kya add kiya: generate_embedding(text) method

```
# Ye method text leti hai, Gemini embedding model se vector banati hai
# Returns: List[float] - 1536 numbers ka array
embedding = await OpenAIService.generate_embedding("Patient ne pain ki complaint ki")
# -> [0.023, -0.145, 0.891, ...] (1536 values)
```

Kyun: Pehle server khud embedding nahi bana sakta tha, frontend ko manually bhejna padta tha.

■ Change 2 — services/ai_automation.py

Kya add kiya: 2 nayi methods

```
# 1. RAG Retriever - query se similar chunks dhundta hai
AIAutomationService.search_similar_chunks(
    db=db,
    query_embedding=[...],      # Embedded query
    organization_id=org_id,     # Sirf is clinic/org ke chunks
    top_k=5,                   # Top 5 results
    document_id=None           # Optional filter
)

# 2. List chunks - ek document ke saare chunks
AIAutomationService.get_chunks_by_document(db=db, document_id=doc_id)
```

■ Change 3 — schemas/ai_automation.py

Kya add kiya: 3 nayi Pydantic schemas

```
KnowledgeSearchRequest    # Frontend se aata hai (query + filters)
KnowledgeSearchResultItem # Ek search result (chunk + score)
KnowledgeSearchResponse   # Poora response (results ka list)
```

■ Change 4 — api/ai.py

Kya add kiya: 2 nayi endpoints + auto-embedding

Auto-embedding — Old vs New

```
# PEHLE: Frontend ko embedding bhejni padti thi
POST /ai/knowledge/chunks
Body: { document_id, organization_id, content, embedding: [0.023, -0.145, ...] } # <- Hard!

# AB: Sirf content bhejo, server baaki karta hai
POST /ai/knowledge/chunks
Body: { document_id, organization_id, content } # <- Easy!
```

■ API Reference — Frontend Developer Ke Liye

Base URL: `https://your-backend.com/ai`

Auth: Authorization: Bearer <JWT_TOKEN> — sab requests mein

API 1 — Document Register Karo (Admin Only)

POST `/ai/knowledge/documents`

Request Body:

```
{
  "organization_id": "uuid-here",
  "title": "Clinic Payment Policy",
  "source_type": "manual",
  "metadata_info": { "author": "Dr. Ahmed", "version": "1.0" }
}
```

Response:

```
{
  "id": "doc-uuid",
  "title": "Clinic Payment Policy",
  "organization_id": "uuid-here",
  "source_type": "manual",
  "created_at": "2026-09-27T14:00:00Z"
}
```

Required Role: super_admin, org_admin, clinic_manager

API 2 — Text Chunk Store Karo (Admin Only)

POST `/ai/knowledge/chunks`

Request Body:

```
{
  "document_id": "doc-uuid",
  "organization_id": "org-uuid",
  "content": "Hamari clinic ka payment policy ye hai ke..."
}
```

■■ **embedding** field mat bhejo — server khud generate kar lega!

Response:

```
{
  "id": "chunk-uuid",
  "document_id": "doc-uuid",
  "organization_id": "org-uuid",
  "content": "Hamari clinic ka payment policy ye hai ke...",
  "embedding": null,
  "created_at": "2026-09-27T14:01:00Z"
}
```

Required Role: super_admin, org_admin, clinic_manager

API 3 — Document Ke Chunks List Karo (Admin Only)

GET `/ai/knowledge/chunks/{document_id}`

Response:

```
[
  {
    "id": "chunk-uuid-1", "content": "Pehla paragraph...",
    "created_at": "2026-09-27T14:01:00Z"
  },
  {
    "id": "chunk-uuid-2", "content": "Doosra paragraph...",
    "created_at": "2026-09-27T14:01:05Z"
  }
]
```


API 4 — ■ RAG Search (Koi Bhi Logged-In User)

POST /ai/knowledge/search

Request Body:

```
{
  "query": "Payment plan kya hai?",
  "organization_id": "org-uuid",
  "top_k": 5,
  "document_id": null
}
```

Field	Type	Required	Description
query	string	Yes	User ka natural language sawal
organization_id	UUID	Yes	Clinic/org ID (current user ki org se lo)
top_k	int (1-20)	No	Kitne results chahiye (default: 5)
document_id	UUID	No	Sirf ek document mein search karna ho to

Response:

```
{
  "query": "Payment plan kya hai?",
  "total_results": 3,
  "results": [
    {
      "chunk_id": "uuid-1", "document_id": "doc-uuid",
      "content": "Hamari clinic installment plan offer karti hai...",
      "similarity_score": 0.0821
    },
    {
      "chunk_id": "uuid-2", "document_id": "doc-uuid",
      "content": "Payment ke liye advance 20% dena hoga...",
      "similarity_score": 0.1543
    }
  ]
}
```

■ similarity_score samajhna:

0.0 = Perfect match (bilkul same meaning) | 0.5 = Related | 1.0+ = Not relevant | **Lower = Better!**

■■ React Frontend — Implementation Guide

Step 1: Admin Panel — Document + Chunks Upload

```
// AdminKnowledgeUpload.jsx
const uploadDocument = async (orgId, title) => {
  const token = localStorage.getItem('access_token');

  // Step 1: Document create karo
  const docRes = await fetch('/ai/knowledge/documents', {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${token}`,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
      organization_id: orgId,
      title: title,
      source_type: 'manual'
    })
  });
  const doc = await docRes.json();
  return doc.id; // document_id save karo
};

const uploadChunk = async (documentId, orgId, content) => {
  const token = localStorage.getItem('access_token');

  // Step 2: Chunk upload - sirf content bhejo!
  await fetch('/ai/knowledge/chunks', {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${token}`,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
      document_id: documentId,
      organization_id: orgId,
      content: content // Server embedding khud banayega
    })
  });
};

// Bade document ko chunks mein split karna
const splitIntoChunks = (text, chunkSize = 500) => {
  const sentences = text.split('. ');
  const chunks = [];
  let currentChunk = '';

  for (const sentence of sentences) {
    if ((currentChunk + sentence).length > chunkSize && currentChunk) {
      chunks.push(currentChunk.trim());
      currentChunk = sentence;
    } else {
      currentChunk += sentence + '. ';
    }
  }
  if (currentChunk) chunks.push(currentChunk.trim());
  return chunks;
};
```

Step 2: Search / Chat Component

```
// KnowledgeSearch.jsx
const searchKnowledge = async (query, orgId) => {
  const token = localStorage.getItem('access_token');

  const res = await fetch('/ai/knowledge/search', {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${token}`,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
      query: query,
      organization_id: orgId,
      top_k: 5
    })
  });

  const data = await res.json();
  return data.results; // [{chunk_id, content, similarity_score}]
};

// Usage in component
const handleSearch = async () => {
  const results = await searchKnowledge(userQuery, currentUser.organization_id);

  // Results display karo
  setSearchResults(results);

  // Ya AI ko context dene ke liye results ka content use karo:
  const context = results
    .filter(r => r.similarity_score < 0.5) // Sirf relevant results
    .map(r => r.content)
    .join('\n\n');

  console.log('Context for AI:', context);
};
```

Step 3: Full Flow Diagram (Frontend ka perspective)

Admin Flow:

[Text Input] -> splitIntoChunks() -> loop -> POST /ai/knowledge/chunks -> OK

User Search Flow:

[Search Bar] -> POST /ai/knowledge/search -> results display

|
context banao from results
|
AI chat mein inject karo

■ ■ Important Notes

- **Similarity Score Threshold:** `similarity_score < 0.5` wale results hi relevant samjho. Isse zyada score ka matlab chunk related nahi hai.
- **Chunking Strategy:** Ek chunk maximum 500-1000 words ka hona chahiye. Bohat bada chunk embedding quality kharab karta hai.
- **Organization Isolation:** `organization_id` hamesha current logged-in user ki org ID se lo — dusri org ke chunks kabhi nahi milenge kyunki backend filter karta hai.
- **Rate Limiting:** Embedding generation ke liye Gemini API calls hoti hain. Bahut saare chunks ek saath upload karne se rate limit ho sakta hai. Frontend mein queue/delay lagao.